almost a natural consequence of designing a container for strings.

Table 6.1 API and contract for StringContainer

Abstract Data Structure: String Container	

API	`class StringContainer {`
	`  insert(key)`
	`  remove(key)`
	`  contains(key)`
	`  longestPrefix(key)`
	`  keysStartingWith(prefix)`
	`}`

Contract with client	Besides all the operations of a regular, plain container, this structure allows to search for the longest prefix of a string that is stored in it, and return all the strings stored that start with a certain prefix

Now that we have fixed an API and described the ADT that we will use to solve our “spell check” problem, however, we are ready to delve into more details and see a few concrete data structures that could implement this ADT.

6.2 Trie

The first implementation of strings container that we will illustrate is the *trie*; incidentally, all the other data structures that we will show in the next sections are based on tries, so we couldn’t choose to start anywhere else.